



A deep learning-based vulnerability detection in blockchain smart contracts using masked attention and control flow graph analysis

Maëlle Gautrin^{1,2} · Arash Habibi Lashkari¹ · Sepideh HajiHosseinKhani¹

Received: 28 June 2024 / Accepted: 29 October 2025 / Published online: 15 November 2025
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2025

Abstract

SCs are self-executing programs on the blockchain, used for transactions without intermediaries, particularly in cryptocurrencies like Ethereum. However, they are vulnerable to security flaws that can lead to significant financial losses, as demonstrated by the DAO hack 2016. Common vulnerabilities include re-entrancy errors, timestamp dependency, infinite loops, and integer overflows. Detecting these flaws is crucial but complex due to the immutable nature of the blockchain and the complexity of the contracts. Therefore, developing techniques for analyzing, testing, and verifying the security of SCs is essential to ensure their reliability and safety. This work presents a novel approach to detecting vulnerabilities in SCs using deep learning. Our SCs vulnerability detection method leverages expert knowledge, masked attention, and deep learning to analyze bytecode. Key steps include constructing a Control Flow Graph (CFG) from bytecode, normalizing the graph by cleaning nodes to retain only opcode instructions, and applying a masked attention mechanism to identify and prioritize critical opcodes. The different graphs obtained are vectorized using Graph2Vec, preparing them for NN processing. Finally, a Recurrent Neural Network (RNN) detects vulnerabilities efficiently and accurately in SCs. The proposed model's performance is evaluated by dataset BCCC-VolSCs-2023 using precision, recall, and F1-score metrics. These metrics collectively offer a comprehensive understanding of the model's effectiveness, emphasizing the quality, completeness, and accuracy of its positive predictions.

Keywords Blockchain · Deep learning · Smart contracts (SCs) · Vulnerability detection

1 Introduction

Smart Contracts (SCs) are autonomous, self-executing programs that enforce and facilitate digital agreements without the need for intermediaries [1]. Their ability to automate and secure transactions has driven the emergence of decentralized applications (DApps) and blockchain-based financial systems [2]. However, this transformative potential is accompanied by significant risks. Once deployed to a blockchain, SCs become immutable, meaning they cannot be altered or patched even if critical vulnerabilities are discovered post-deployment [3]. As a result, any flaw embedded in the contract code can lead to irreversible and potentially catas-

trophic consequences, such as unauthorized fund transfers or permanent denial of service.

Detecting vulnerabilities in SCs has become an urgent and complex challenge. Despite being written in high-level languages like Solidity and deployed on secure platforms like Ethereum, SCs often suffer from exploitable issues. These include reentrancy attacks [4, 5], which enable malicious recursive calls; timestamp manipulation [6, 7], which exploits the block timestamp to alter contract behavior; integer overflows [8, 9], which cause miscalculations; and infinite loops [10], which deplete gas and render contracts unresponsive. These vulnerabilities are technically sophisticated and difficult to detect due to the SCs' execution semantics and the diversity of coding practices across the ecosystem.

Multiple representations have been used in past research to detect these vulnerabilities, ranging from raw source code to opcodes, control flow graphs (CFGs), and hybrid forms. While deep learning and graph-based models have shown promise, the field still faces key challenges: the complexity of modeling SC behavior, limited transparency in model decisions (especially in black-box architectures), inconsis-

✉ Sepideh HajiHosseinKhani
sepid99@yorku.ca

¹ Behavior-Centric Cybersecurity Center (BCCC), School of Information Technology, York University, Toronto, Ontario, Canada

² Department of Computer Science, ENS Paris-Saclay, University Paris-Saclay, Gif-sur-Yvette, France

tent access to high-quality labeled datasets, and the inability to generalize across diverse contract formats. These issues hinder practical adoption and highlight the need for interpretable, efficient, and robust detection mechanisms.

Furthermore, researchers have utilized various representations of SCs, such as source code, opcodes, and graphs, to detect these vulnerabilities. The primary representation is source code, which is directly used to identify vulnerabilities and develop new strategies for managing emerging risks. By combining deep learning and machine learning techniques with source code-based methods, researchers have developed tools such as DeeSCVHunter [11], SPCBIG-EC [12], and Clear [13]. These tools are notable for their innovative approaches, significantly enhancing the accuracy and efficiency of vulnerability detection, but they remain constrained by the availability of source code.

Consequently, some research focuses on utilizing opcodes, which provide clear instruction sequences for enhanced precision. This approach is exemplified in the works of Jain et al. [14] and the use of ESCORT by Sendner et al. [15]. Another representation used in recent research is graph-based methods. These methods, such as those employed by Chen et al. in GasChecker [16] and Zhen et al. in DA-GNN [17], focus on control and data flows, offering clear visualizations of SC structures and improving the identification of vulnerabilities. Finally, hybrid approaches integrating multiple representations are also being explored to optimize vulnerability detection. For instance, Liu et al. [18, 19] and Zhang et al. [20] utilize, respectively, source code with graph and bytecode with opcodes representations to address more aspects of SCs than a single representation can capture.

SCs' vulnerability detection presents numerous challenges, underscored by past research. These challenges, such as complexity, limited transparency, accessibility, and rigidity of the models, persist, hindering the full potential and broader adoption of these methodologies. To tackle some of these challenges, we propose a novel approach based on graph theory. Our method combines expert knowledge, masked attention mechanisms, and deep learning techniques to extract the information from bytecode. This method includes different steps: constructing a CFG to delineate the contract's operational pathways, standardizing the graph to emphasize opcode instructions, implementing masked attention to spotlight critical facets, transforming subgraphs into vectors for NNs processing, and employing a Recurrent Neural Network (RNN) for vulnerability detection. This model empowers us to discern vulnerabilities precisely and efficiently by exploiting the intricate details embedded within the bytecode.

Furthermore, we used the dataset BCCC-VoISCs-2023 for our experiments and to evaluate our model. This dataset contains 34,943 SCs, including 25,903 secure and 9040 vulnerable contracts. The results obtained from these experiments,

which evaluate precision, recall, accuracy, and F1-score metrics, prove the performance of our model and the interest of the masked-attention mechanism in detecting multiple vulnerabilities.

Our study brings diverse contributions, marking substantial strides in vulnerability detection. These contributions include:

- Design and implement a novel approach to vulnerability detection that leverages graph-based techniques. The proposed model can detect potential vulnerabilities in SC by representing the relationships between SC components as a graph.
- Develop a novel preprocessing technique explicitly tailored to our graph-based model. This preprocessing step is designed to optimize data quality and relevance, thereby improving the accuracy and reliability of the proposed detection system.
- Propose a new taxonomy for categorizing SC methodology based on the representations used in vulnerability detection. This taxonomy offers a structured framework for understanding and classifying different types of vulnerabilities in SCs, facilitating more comprehensive analyses and targeted mitigation strategies.

The rest of this paper is structured as follows: The Sect. 2 provides an in-depth study of SC and essential contextual information. The Sect. 3 reviews the existing literature in this research area and highlights the limitations inherent in current solutions. The Sect. 4 comprehensively introduces the proposed approach and describes its conceptual framework and implementation in detail. The Sect. 5 provides a comprehensive overview of the extensive experiments conducted, along with the obtained results and the implementation details of our model. The Sect. 6 critically analyzes and discusses the research questions and highlights the implications of our findings. Finally, the Sect. 7 presents the conclusions we draw from our study and suggests avenues for future research and development.

2 Background

This section provides a comprehensive background on blockchain technology, SCs, and their associated vulnerabilities. It begins with Sect. 2.1, which covers the fundamentals of blockchain technology, explaining its mechanisms and relevance. Next, Sect. 2.3 delves into SCs, including their definition, functionality, and applications. Finally, Sect. 2.2 examines the vulnerabilities inherent in SCs, discussing potential security risks and weaknesses. This structured approach aims to build a solid foundation for understanding the necessity and advantages of the proposed model.

2.1 Blockchain

Blockchain technologies like Ethereum have revolutionized digital interactions by providing a secure and reliable platform for various applications [21]. Their distinctive features are particularly effective in fostering trust and ensuring security in digital transactions [22]. Blockchains operate on a transparent ledger, where all transactions and data are publicly accessible [23]. The integrity of this data is safeguarded by cryptographic algorithms, which prevent unauthorized access and manipulation [24]. Once recorded, data on a blockchain is immutable, meaning it cannot be altered or deleted [25]. This capability can potentially revolutionize various industries, including financial services and supply chain management [26]. In conclusion, the unique characteristics of blockchain technology make it a powerful environment for enhancing security and trust across numerous sectors.

2.2 Smart contracts

SCs are self-executing computer programs that run on Ethereum blockchain platforms, operating automatically according to predetermined logic [2]. They represent a significant advancement in blockchain 2.0, enabling the execution of a wide range of applications beyond simple cryptocurrencies like Bitcoin [3]. SCs are typically written in high-level languages like Solidity, which are then compiled into EVM bytecode and executed in the Ethereum Virtual Machine (EVM) [27].

Due to their numerous advantages, such as automatic execution, the elimination of trusted intermediaries, and strong resistance to tampering, SCs have the potential to revolutionize various sectors, including retail banking, insurance, financial exchanges, marketplaces, and content platforms [28]. With their transformative capabilities and growing adoption, SCs are set to reshape the landscape of modern business and transactions [1]. In conclusion, integrating SCs in blockchain technology marks a pivotal shift, offering enhanced efficiency, security, and automation across multiple industries.

2.3 Different types of vulnerabilities

While SCs offer significant benefits in automating and securing transactions—such as supply chain management, real estate, and digital identity verification—their immutable and self-executing nature introduces unique vulnerabilities that can be exploited, potentially causing significant issues [29]. In the following, we outline four categories of SC vulnerabilities of particular concern: re-entrancy, timestamp manipulation, integer overflow, and infinite loops [30]. These vulnerabilities were selected not only because they repre-

sent some of the most common and damaging weaknesses in SCs (as demonstrated by high-profile incidents like the DAO attack [4]), but crucially because they originate from fundamentally different contexts, spanning state management, time dependency, arithmetic operations, and control flow, respectively. This deliberate diversity in vulnerability types will allow us to evaluate our approach thoroughly. Although other vulnerability types exist, these four provide comprehensive coverage of the major vulnerability patterns while being representative enough to draw meaningful conclusions about detection effectiveness. Addressing them is crucial to ensure the reliability, security, and trustworthiness of SCs, highlighting the importance of thorough auditing and rigorous testing in SC development.

2.3.1 Re-entrancy

Re-entrancy vulnerability refers to a situation where a malicious contract can call back into the calling contract before the first invocation is completed. These attacks transpire when an assailant depletes funds from the target by repeatedly invoking its withdraw function. Should the contract neglect to update its state regarding a victim's balance before transferring funds, the attacker can persistently call the withdraw function to exhaust the contract's funds [5]. One notable real-world instance of a re-entrancy attack is the DAO attack, which lost 60 million US dollars. According to Samreen et al. in [4], the most damaging assaults on Solidity SCs are re-entrancy vulnerabilities.

2.3.2 Timestamp

Timestamp dependency is a vulnerability in SCs that arises when they rely on the timestamp of a block to trigger critical actions, such as sending funds or distributing rewards [6]. Malicious miners can exploit this vulnerability by manipulating the block's timestamp to alter the contract's outcome [7]. According to Luu et al. in [6], developers can enhance contract security by incorporating randomness into the contract's logic, implementing timelocks, and employing robust consensus mechanisms. Additionally, regularly auditing and monitoring SCs helps protect users from potential exploits.

2.3.3 Integer overflow

The integer overflow vulnerability occurs when a mathematical operation on an integer exceeds its maximum storage capacity, causing it to wrap around to zero or transition to negative values [9]. According to Lai et al. in [8], this can lead to fund losses or unexpected behaviors. To mitigate this issue, developers should check value limits and use appropriate data types, and security audits are crucial for detecting and rectifying integer overflow vulnerabilities.

2.3.4 Infinite loop

The infinite loop vulnerability occurs when code includes an iteration or loop that lacks a defined exit condition or has an exit condition that cannot be reached [19]. According to Zhuang et al. in [10], in SCs, the fallback mechanism introduces a new avenue for this non-termination bug, particularly through cyclic calls between functions and the fallback function.

SCs' security plays a crucial role in the security of blockchain. While they offer significant benefits in automating and securing transactions, their inherent characteristics also introduce unique risks that malicious actors can exploit. Various vulnerabilities, such as reentrancy, timestamp, integer overflow, and infinite loop, have been identified as common vulnerabilities of SCs. As highlighted by previous research, attackers can exploit these vulnerabilities, leading to financial loss. The reentrancy vulnerability is located in the call of the withdraw function, timestamp in the presence of the block's timestamp, an integer overflow in the arithmetical operations, and an infinite loop in the exit condition for the iterations or loops. Developers can mitigate these vulnerabilities through careful design, thorough auditing, and robust security measures, ensuring SC-based systems' reliability, security, and trustworthiness in blockchain technology.

3 Related works

This section reviews pertinent works and summarizes current methods for detecting vulnerabilities. In addition, we provide a thorough taxonomy (Fig. 1) to characterize and group these techniques according to the SC representations they employ, including source code, opcode, n-grams, images, and graphs. The last sub-section, synthesis, concludes the gaps and shortcomings in the body of literature and briefly explains how our proposed model responds to most of them.

3.1 Current models

In this section, we comprehensively review relevant literature and present an overview of the current methodologies employed for detecting vulnerabilities in SCs. To facilitate a structured understanding, we introduce a detailed taxonomy, as depicted in Fig. 1, which categorizes and organizes these techniques based on the representations of SCs they utilize. These representations encompass source code, opcode sequences, n-grams, images, and graphs.

3.1.1 Source code

The first category is source code, which is the most common representation due to its frequent use in previous works [11–

13, 31–36]. Analyzing source code enables the detection of potential vulnerabilities by leveraging natural language processing techniques.

Yu et al. [11], Wu et al. [35], and Zhang et al. [36] deal with the loss of generality when relying on expert rules and knowledge in source code. Yu et al. in [11] DeeSCVHunter aims to identify critical vulnerabilities using Vulnerability Candidate Slices (VCS) in the source code, with a special focus on re-entrancy and timing-related vulnerabilities. The modular design of DeeSCVHunter includes various deep-learning models and embedding schemes to achieve robust and explainable vulnerability detection without relying on expert knowledge. They conducted experiments on real datasets, and the results demonstrated the effectiveness of VCS, with an F1 score improvement of up to 25.76%.

Wu et al. in [35] combines expert advice with smart computer tricks to find vulnerabilities thoroughly without relying on expert rules, which have poor scalability and high false negative and false positive rates. They propose a hybrid attention mechanism model that extracts code fragments from the source code, focusing on key vulnerability points. It allows us to consider the semantic information and the context of the source code. Zhang et al. in [36] combines with CBGRU different computer techniques to stop relying too heavily on complex rules defined by experts, significantly increasing the detection time. The model extracts features from the source code through different deep-learning models and combines these features for SC vulnerability detection. Through a series of experiments, they demonstrate that the CBGRU hybrid model has great SC vulnerability detection performance on the SmartBugs Dataset-Wild.

Furthermore, Torres et al. [37] proposed HONEYBADGER, a tool that detects honeypot issues in SCs by executing the code and analyzing EVM bytecode as input, producing a detailed report on the various honeypot issues it identifies. Chen et al. [38] developed GASPER, which explores all possible paths in the contract's code to detect inefficient coding by assessing gas efficiency through bytecode analysis of SCs. Moreover, Liu et al. [39] developed ReGuard, a fuzzing-based tool to detect reentrancy vulnerabilities in Ethereum SCs by analyzing their C++ source code version.

Alternatively, Chen et al. [13], and Yang et al. [34] use source code words embedding in a machine learning model to detect vulnerabilities in SC. Chen et al. in [13] propose a novel approach, Clear, which captures the fine-grained correlation information among source code contracts and generates correlation labels based on the relationships between contracts to guide the training process of the contrastive learning model to see how contracts fit together. Clear is evaluated through an empirical evaluation of a large-scale real-world dataset, which shows that Clear achieves optimal performance compared to baseline methods.

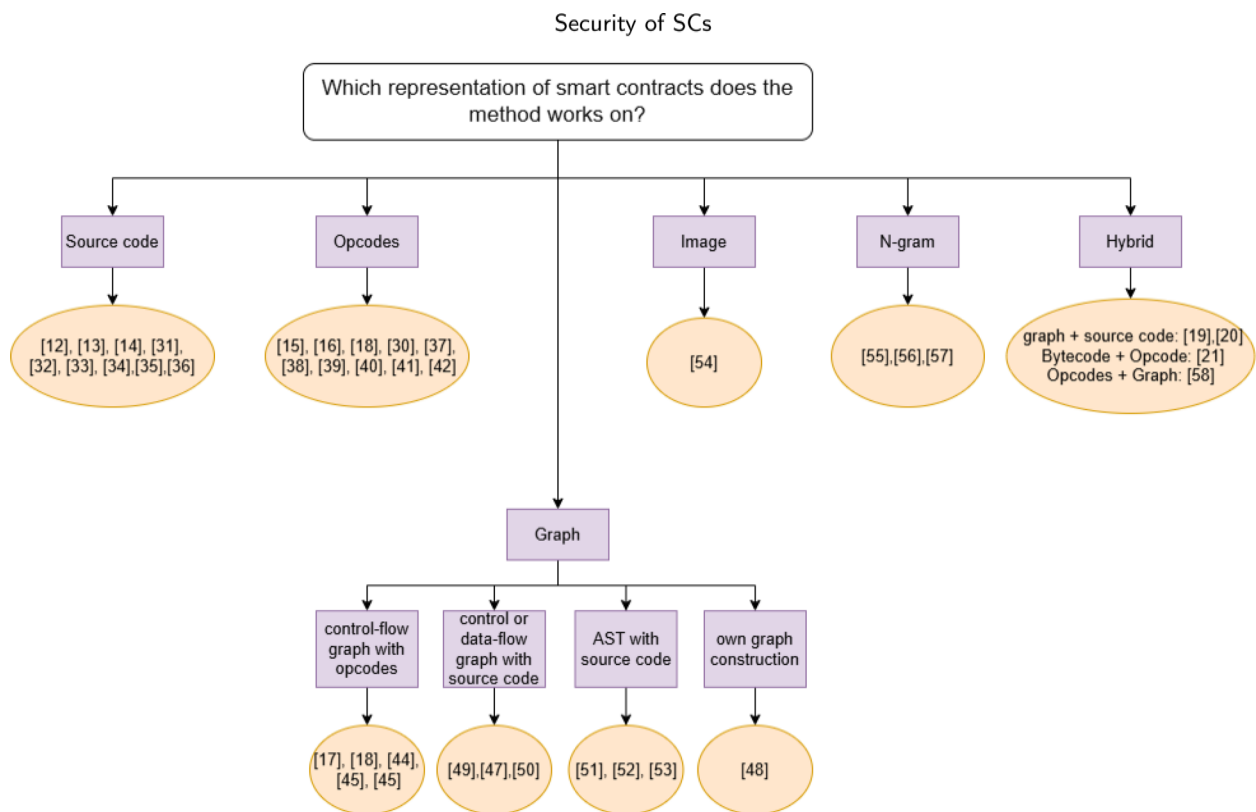


Fig. 1 Taxonomy

Moreover, Yang et al. propose two different methods to detect vulnerabilities with source code in [34]. SCSVM uses a word-to-vector technique to extract the features of Ethereum SCs source code using support vector machine technology and combines it to identify vulnerabilities. Experiments on Smartbugs and Smartbugs-wild show that SCSVM is significantly effective. SCLMF alleviates the problems of deep learning methods over-relying on large-scale data to train models and collecting a large number of SC attack samples in a short period by proposing a basic learner-meta-learner framework containing a preprocessing of source code using the word embedding technique before being transformed into word vector form as an input to the SVM model. After computing the loss with a cross-entropy loss function, the vulnerability detection findings are derived by applying a decision function to categorize the vulnerabilities. The experiments on WScrawlID show that SCLMF has a certain detection effect.

In another light, Sun et al. [31], and Yiting et al. [33] optimize the detection of vulnerabilities in the SC source code using pre-trained deep-learning model. Sun et al. in GPTScan [31] aims to reduce the number of false positives that appear by relying solely on GPT to identify vulnerabilities. It combines the Generative Pre-training Transformer (GPT) model with analyzing the source code itself, which helps it find

tricky vulnerabilities in SCs with great accuracy. Evaluation of diverse datasets shows that GPTScan achieves high precision for token contracts by reducing the number of false positives. Yiting et al. propose a BERT pre-training model in [33] to quickly and effectively process and detect SCs, making it better at picking out important information without strict rules. It preprocesses and makes symbol substitutions in the source code contract, which makes the pre-training model better obtain contract features. They evaluate the method on four datasets and compare its accuracy with other deep learning models, demonstrating its superiority.

Last, Lejun et al. [12] and Guo et al. [32] present two other deep-learning methods. Lejun et al. proposed the SPCBIG-EC model in [12]. It uses advanced deep learning techniques in a Serial-Parallel Convolution setup designed to find vulnerabilities in IoT devices connected to blockchain technology. It showed excellent performance benefits in SC vulnerability detection by extracting features from the input sequence for multivariate combinations while retaining the source code's temporal structure and location information. Guo et al. in [32] use the multi-scale cascade encoder architecture to hit well-known high-risk vulnerabilities in SCs. They enhance source code feature semantics with a unique Surface Feature Encoder, utilize a dual-branch encoder to capture global and local features, and employ the Deep Residual Shrink-

age Network to focus on vulnerability-related characteristics. Experimental results on three types of high-risk vulnerability datasets demonstrate performance compared to state-of-the-art methods, and our method achieves an average detection accuracy of 90%.

As observed in the recent approach studied in [11–13, 31–36], a method based on source code can show an improvement of the accuracy in the way it considers the context of each element of the SC, and in the time needed to perform the detection. Despite this improvement, the source code still has the problem of availability. Therefore, some other studies prioritize different representations of SCs.

3.1.2 Opcodes

The second category is opcodes, another representation of SCs that detect vulnerabilities in [14, 15, 17, 30, 40–45]. These methods use opcode representations directly to strengthen SC vulnerability detection, deal with the difficulties that come with it, and make the analysis more accurate. Regarding the availability, opcodes can be obtained from bytecode, which contains all the necessary information to run the code, by decomposing it into instructions and identifying the opcodes associated with each instruction [46].

On top of that, Sui et al. [41], Huang et al. [42], and Sendner et al. [43] focus on tackling the problem of bytecode diversity by basing the vulnerability detection on generic opcodes. In [41], Sui et al. combine static analysis with symbolic execution; this method finds vulnerabilities in a detailed way by proposing the expansion and improvement of the internal module of Mythril, which realizes the operation of automatic vulnerability analysis when performing vulnerability detection by improving the operation efficiency and simplifying the operation of SC opcodes while analyzing them. The comparison of experimental results shows that the proposed method is more suitable for SC vulnerability detection, improving accuracy and efficiency.

Furthermore, Huang et al. Propose another method based on a hard-sharing design using opcodes as representation in [42]. It deals with the challenges of different bytecode styles and messy code by cleaning up the bytecode, making it easier to match with opcodes and find vulnerabilities. It uses a bottom-sharing layer to learn the semantic information of the input contract, and then a task-specific layer is employed to realize the functions of each task. A classical Convolutional Neural Network is used to construct a classification model for each task that learns and extracts features from the shared layer for training to achieve their respective task objectives. The experimental results show that the model can better identify the types of vulnerabilities after adding the auxiliary vulnerability detection task. Sendner et al. propose, in [43], ESCORT, a tool based on deep learning that uses a standard feature extractor to learn generic opcode semantics

of SCs and separate branches to learn the features of each vulnerability type. ESCORT is the first deep learning-based framework that utilizes transfer learning on new vulnerability types with minimal model modification and re-training overhead. Compared with existing non-ML tools, ESCORT can be applied to contracts of arbitrary complexity and ensures 100% contract coverage.

However, Jain et al. [14], Liu et al. [15], and Huang et al. [30] propose transformer-based detection at the opcode level. In [14], Jain et al. suggest a hierarchical model that uses deep learning techniques. This research proposes a two-step hierarchical model with a transformer model to look at how opcodes are related, picking out important information to understand the context better, and a Bi-GRU to aggregate forward and backward sequential information for long-term reliance. These steps significantly improve the feature extraction mechanism for Ethereum SCs, as proven by the experiment conducted on a created dataset of SCs, demonstrating that the proposed method is more effective than state-of-the-art methods.

Along with that, Liu et al. propose DL4SC in [15], a novel deep learning-based vulnerability detection at the opcode level. It combines the Transformer encoder and CNN to detect vulnerabilities of SCs and exploits SSA (Sparrow Search Algorithm) to search model hyperparameters for vulnerability detection automatically. The model addresses the challenges of the other deep learning-based methods, which have low accuracy, are time-consuming, and have an application range that is too small. Huang et al. suggest dealing with the diversity of opcode in [30] and conducting data- and instruction-level normalization to uniform it. Their method encodes an unsupervised graph embedding algorithm, which combines opcode sequences into a new vector representation after word embedding and positional encoding. The vector is then input to the Transformer encoder to assign weights to the vulnerability features and obtain global feature information. The evaluation shows the efficiency and effectiveness of the model and the interest in considering the diversity of opcodes by compilers in detecting vulnerability.

Continuing in the opcode category, Sun et al. [40] and Mezina et al. [44] propose to deal with opcode representation using the machine learning method. Sun et al. propose an approach using machine learning methods in [40]. This approach aims to build a general benchmark for new vulnerability detection while trying to standardize opcodes to reduce the demand for expert manpower. This method uses fast machine learning algorithms to find vulnerabilities quickly and can be used as a fast prefilter for the traditional symbolic analysis tools to improve accuracy. Mezina et al. in [44] overpasses the challenge of preserving performance while maintaining a decent level of security in a constantly increasing number of contracts. This work proposes a new ML-based two-phase approach for detecting and classifying vulnera-

bilities in SCs. The opcodes are encoded with a tokenizer as the preparation step, which helps to represent the given opcodes as integer sequences for the vulnerability detection part. Results show an accuracy of 0.9921, F1 score of 0.9902, precision of 0.9883, and recall of 0.9921 within a reasonable execution time, which could be suitable for integrating existing DLTs.

Opcode utilization demonstrated its performance through various methods such as similarity comparison [30], machine learning [40, 44], and deep learning [14, 15, 20, 43]. These methods proved to be as effective as source code-based approaches. However, the lack of focus on precise information hindered the potential enhancement of performance accuracy.

3.1.3 Graph

Moving forward to the next category, graph-based methods are becoming more frequent [16, 17, 47–56]. Using opcode or source code to be constructed allows a clear visualization of the SC structure, highlighting the flow of information. The first type of graph is the CFG based on opcode. Several methods use it to improve the analysis and find potential security vulnerabilities [16, 17, 47–49]. Among these methods, Chen et al. present GasChecker in [16], which summarizes ten gas-inefficient programming patterns and proposes a new approach that parallelizes Symbolic by tailoring it to the MapReduce programming model to find patterns in the CFG. Extensive experiments show that GasChecker scales well with the increase in workers.

Chang et al. propose sCompile in [47], an alternative approach to automatically identify critical program paths in an SC and rank the paths of the CFG according to their criticalness. It identifies paths involving monetary transactions as critical paths and prioritizes those potentially violating essential properties. Symbolic execution techniques are then applied to top-ranked critical paths. This tool has been evaluated on 36,099 SCs, and it shows that many known vulnerabilities can be captured if the user inspects as few as 10 program paths generated by sCompile. Similarly, in [48], Chen et al. analyze the characteristics of vulnerabilities in SCs and the corresponding protection strategies by establishing a set of detection rules based on semantic facts, taking the Datalog facts extracted from the CFG and querying the database using defined declarative inference rules. The experimental results of the evaluation on real Ethereum SCs show that SafeCheck performs better than other typical tools.

Additionally, Zhen et al. in [17] put forward DA-GNN (Dual Attention Graph Neural Network), which converts the operation code sequence of SC CFG nodes into a feature matrix of semantic features and relationships. It uses a dual attention mechanism to integrate these features into the

Graph Attention Network for node embedding updates. The updated node information is combined using a self-attention mechanism to obtain graph features. They evaluate DA-GNN on a new dataset, which proves they succeeded in addressing the insufficient feature extraction in other NN-based approaches for SCs. In Guardian ADRGAN [49], Pise et al. implement a feature extraction from the CFG using Nested patch-based feature extraction. Then, extracted features are fed to an attentive dual residual generative adversarial network, identifying the vulnerabilities of the K framework's Ethereum virtual machine SCs. The simulation outcomes prove that the proposed ADRGAN-SCB-KEVM technique attains higher Accuracy for Vulnerable and Non-Vulnerable compared to the existing methods.

The second type of graph is the control or data flow graph constructed with source code used by Wu et al. [52], Zeng et al. [50]. Liu et al. in [53], it has the same properties as the CFG with opcode but conserves the natural language behind this. In EtherGIS [50], Zeng et al. adopt Graph Neural Network (GNN) to aggregate the whole control and data flow graph's attribute and structure information, bridging the semantic gap between low-level graph features and high-level contract features. The feature representation of the graph is then input into the contract graph classification model for vulnerability. They build up a dataset for evaluation, and results demonstrate that EtherGIS can productively detect vulnerabilities in Ethereum SCs regarding accuracy, precision, recall, and F1-score detection.

Wu et al. in the Peculiar's method [52] uses a data flow graph which is less complex than the CFG representation used in [16, 17, 47–51] and does not bring an unnecessarily deep hierarchy to represent the SC which makes the model easy to focus on the critical features. This data flow graph is applied to the pre-training technique due to its dramatic improvements on various NLP tasks. The experiment shows that Peculiar is more discerning of re-entrancy vulnerability than existing approaches. Liu et al. in [53] combine a GNN model from the data and CFG with enhanced detection efficiency and stronger generalization, alongside an expert model, to improve the screening of SCs for specific vulnerabilities. This hybrid approach mitigates the interpretability issues of deep learning while compensating for the limitations of expert rules. This method significantly improves vulnerability detection effectiveness, scoring approximately 6 points higher on average than using the GNN model alone.

Another type of graph is used in [51]. Indeed, in CVDIE-ENSEMBLE, Zhang et al. constructed their graph to highlight how information and parts of the code are related. The method is based on seven different NNs pre-trained using an information graph consisting of source datasets, which were then integrated into an ensemble model called the SC Vulnerability Detection method based on Information Graph and Ensemble Learning (SCVDIE). The verification and com-

parison results show that the proposed SCVDIE method has higher accuracy and robustness than other data-driven methods.

The last type of graph is Abstract Syntax Trees (ASTs) as used in [54–56]. AST is a developed version of CFG without a cycle, which allows clear access to the different paths of the program. Some methods employ ASTs with deep learning models, as demonstrated by studies such as [54, 55], while others concentrate on analyzing Hyperledger Fabric SCs [56]. This method proves good performance by simplifying the run of the CFG. Moreover, Wu et al. [57] introduced TaintGuard, a symbolic execution-based approach that monitors program code to track changes in the AST during execution, aiming to prevent unauthorized modifications to contract privileges by external actors.

In conclusion, graph-based methods have gained significant popularity for vulnerability detection in SCs due to their ability to represent the structure and flow of information visually. By leveraging CFGs based on opcode and source code, researchers like Chen et al. [16] and Zhen et al. [17] have developed sophisticated techniques that enhance the precision and scalability of detection tools. Methods such as GasChecker and DA-GNN have demonstrated the effectiveness of using CFGs for identifying gas-inefficient patterns and semantic feature extraction, respectively. Additionally, the use of data flow graphs and CFGs by researchers like Wu et al. [52] and Liu et al. [53] has shown improved accuracy and generalization in detecting vulnerabilities, particularly with techniques like pre-training and hybrid models. Furthermore, abstract syntax trees (ASTs) have been utilized to streamline the detection process by providing a more accessible representation of program paths, as seen in various studies. Overall, the integration of graph-based methods with advanced NNs and symbolic execution techniques represents a promising direction for enhancing the robustness and accuracy of SC vulnerability detection.

3.1.4 Image

The next category is image, which is used only by Hwang et al. in [58]. They use an image of bytecode to represent the SC. Traditional methods based on generic CNN architectures fail to properly analyze the semantics and context of SCs, resulting in numerous false positives and false negatives [58]. Implementing CodeNet by Hwang et al. preserves context and semantics by utilizing deep learning after data preprocessing, which consists of transforming the SC into images while maintaining their context using CNN without strides. The proposed method shows good enough detection performance and time compared to well-known state-of-the-art vulnerability detection tools to efficiently and accurately identify vulnerabilities.

In conclusion, the innovative approach by Hwang et al. [58] in transforming SC bytecode into images demonstrates significant advancements in vulnerability detection. By preserving the context and semantics of the SCs through the CodeNet framework, they effectively address the limitations of traditional CNN-based methods, which often struggle with false positives and negatives. This method achieves competitive detection performance and maintains efficient detection times, highlighting its potential as a reliable tool for accurately identifying vulnerabilities in SCs.

3.1.5 N-grams

The last category is the n-grams. They are derived from opcode sequences and can provide access to each opcode separately without losing its context. N-grams are used in recent works as a representation of SCs to detect vulnerabilities in [59–61].

In [59], Mi et al. suggest novel feature vector generation techniques based on opcodes n-grams to extract vectors from each SC. The collected vectors are fed into a novel metric learning-based deep NNs to get the detection result. In evaluating this approach, the quantitative results demonstrate its effectiveness and efficiency. Wang et al. bring ContractWard in [60], which extracts bigram features from simplified opcodes of SCs and then employs five machine learning algorithms and two sampling algorithms like SMOTE and SMOTE-Tomek to build the models based on these vectors. The experimental results demonstrate the effectiveness and efficiency of ContractWard. Similarly, in another study, Li et al. [61] introduce a machine learning-based method for detecting unknown vulnerabilities in SCs. They employ an n-gram model and a vector weight penalty mechanism to extract the opcode sequence features. Then, machine learning algorithms are used to detect unknown vulnerabilities based on the similarity principle in these vectors. The study's experimentation proves this method's advantages, particularly the rapid detection of attacks exploiting unknown vulnerabilities, thereby minimizing user losses.

In conclusion, utilizing n-grams derived from opcode sequences represents a promising avenue for detecting vulnerabilities in SCs. By affording access to individual opcodes while retaining their contextual significance, n-grams offer a robust representation of SCs. Recent studies, such as those by Mi et al. [59], Wang et al. [60], and Li et al. [61], have demonstrated the efficacy of n-gram-based approaches. These methodologies leverage machine learning algorithms and novel feature extraction techniques to identify vulnerabilities in SCs efficiently. The experimental evaluations of these methods consistently highlight their effectiveness in rapidly detecting known and unknown vulnerabilities, thereby mitigating potential user losses.

3.1.6 Hybrid

After exploring the different categories of representation, such as source code, opcodes, graph, n-grams, and image, it is essential to highlight the work of Liu et al. [18, 19], Zhang et al. [20], and Chen et al. [62], who use a hybrid representation of SC that combines elements from multiple categories.

Liu et al. in [18] and [19] integrates expert-defined security patterns based on source code with graph NNs which used the control and data flow semantics of a contract graph with employing a temporal message propagation network to extract graph features and combining them with expert patterns. Experiments conducted on all available SCs with source code in Ethereum and VNT Chain show that this system significantly outperforms state-of-the-art methods. Similarly, in [20], Zhang et al. address the limitations of existing processes by combining machine learning-based vulnerability detection with multi-objective detection algorithms using bytecode and opcode. Utilizing a Multiple-Objective Detection Neural Network (MODNN), this method supports the parallel detection of multiple vulnerabilities and achieves high scalability. Furthermore, Chen et al. propose, in [62], to use feature vectors from opcodes and CFG for the machine learning model training. These data are transformed into feature vectors used to train the Multi-layer Perceptron. The model is evaluated on SCs deployed on the Ethereum Blockchain, where it demonstrates high accuracy and a low false positive rate, making it an effective tool for SC vulnerability detection.

In conclusion, hybrid techniques amalgamating elements from multiple categories, offer promising avenues for enhancing detection accuracy and efficiency. Liu et al. [18, 19], Zhang et al. [20], and Chen et al. [62] have proposed innovative approaches that integrate expert-defined security patterns, machine learning algorithms, and GNN. These methods leverage the synergies between different representation categories to achieve superior vulnerability detection performance. Experimental evaluations across different SC datasets consistently validate the effectiveness and scalability of these hybrid techniques, positioning them as valuable tools in the ongoing effort to enhance SC security.

3.2 Synthesis

In summary, SC vulnerability detection methods utilize various representations, such as source code, opcodes, graphs, images, n-grams, and hybrids. Each approach addresses different aspects of vulnerability detection, employing techniques such as deep learning [11, 12, 14, 15, 17–20, 30–34, 36, 42, 43, 49–53, 55, 58, 59] and machine learning [13, 34, 40, 44, 60–62] or other model [16, 41, 47, 48, 54, 56]. These diverse methodologies collectively aim to enhance the

accuracy and efficiency of SC security analysis. Despite this improvement, it still has issues:

Similarity comparison: Comparing the similarity score of SC and vulnerable SCs may not adequately capture subtle variations or evolving attack vectors, potentially resulting in false positives or overlooking emerging threats [30].

Source-code dependency: Basing the detection on source code utilization leads to a lack of generalization because the source code of SC is rarely available, while bytecode can always be obtained directly from the blockchain [15].

Parallel NNs: Using several NNs to detect different vulnerabilities causes a loss of resource utilization [51].

Pre-defined pattern: Basing the vulnerability detection on a pre-defined pattern may struggle to adapt to novel attack patterns or variations, potentially missing out on emerging threats or failing to detect subtle vulnerabilities [47].

Hard-parameter sharing: It may weaken the expressiveness of the model and reduce the generalization ability of the model [42].

Single slice contract description: Describing the contract by a single slice of the function leads to a loss of generality because a single slice describes the characteristic of greedy contracts [30].

Imprecise results: Giving a binary classification is imprecise compared to giving a probability of vulnerability, the origin of the vulnerability, or the type of vulnerability detected [40].

Opacity of deep-learning: Decisions made by deep neural network models are often hard to understand, compromising interpretability, trust, and the ability to identify and correct biases [12].

Use of symbolic execution: Symbolic execution presents some inherent limitations because of its inability to accurately model certain dynamic and complex behaviors inherent in decentralized systems, leading to potential inaccuracies and incomplete vulnerability assessments [47].

To overcome these first six challenges, precision, effectiveness, and generalization ability need to be improved. In this research, we propose a model combining expert knowledge, attention mechanisms, and deep learning to detect different vulnerabilities in SCs.

4 Methodology

This section discusses the methodology for detecting SCs vulnerabilities. From the synthesis of related works, it has been observed that some critical issues include a lack of precision, effectiveness, generalization, and imprecise results. These issues are partly caused by source code dependency, use of similarity comparison, employment of several parallel NNs, exploitation of predefined patterns, usage of hard parameter sharing, and consideration of single-slice contract

description. This research proposes a method that combines expert knowledge, masked attention, and deep learning to navigate and cover these limitations, extracting meaningful information from bytecode, which is always available from the blockchain, unlike source code.

The novelty of this approach lies in its hybrid design that fuses symbolic graph representations with a masked attention mechanism and a graph-level vectorization pipeline, all operating directly on bytecode-level CFGs. Unlike previous works that depend on source code or static predefined rules, our method integrates a contextual masking strategy that dynamically adjusts the graph encoding based on vulnerability type. This dynamic focus allows selective enhancement of the signal without losing generality. Moreover, the fusion strategy of masked outputs ensures that cross-vulnerability signals are not discarded, a key improvement over previous hard-isolation or over-aggregating models. Figure 2 presents the overall architecture of the proposed model.

The following subsections present the proposed model including CFG construction 4.1, normalization process with node cleaning 4.2, masked attention mechanism to prioritize critical aspects of opcodes 4.3, vectorization to prepare data for the next step 4.4, and vulnerability detection with a Recurrent Neural Network (RNN) 4.5. This model efficiently and accurately detects vulnerabilities in SCs by leveraging the detailed information within the bytecode.

4.1 CFG construction

CFG is a representation of SC that gathers all necessary information about the contract's data flow, highlighting its architecture without losing any detail [63]. It is constructed using opcodes, sequences of instructions extracted from bytecode [64]. The proposed method uses this representation of SCs to detect vulnerabilities with the EtherSolve method [46]. As demonstrated in [46, 63, 64], CFGs extracted from bytecode, despite potential abstraction loss, maintain sufficient structural fidelity to support reliable downstream analysis. These studies reinforce the premise that CFGs are robust representations in static analysis tasks.

The process begins by parsing the raw bytecode into executable code and metadata to generate opcodes. The generated opcodes are then grouped into basic blocks, where each block represents a continuous sequence of opcodes between a jump target and a jump instruction. Instructions that alter control flow, such as JUMP, JUMPI, STOP, REVERT, RETURN, INVALID, and SELFDESTRUCT, mark the end of a basic block, while JUMPDEST marks the beginning of a new one.

Once basic blocks are identified, EtherSolve constructs the CFG edges by resolving the destinations of simple jumps based on preceding PUSH opcodes. Orphan jumps not preceded by a PUSH require symbolic stack execution to determine their destination. This involves simulating stack

behavior using symbols representing unknown values and following specific opcodes (PUSH, DUP, SWAP, AND, and POP). The simulation resolves jump destinations using the symbolic state of the stack.

After building a partial CFG, EtherSolve proceeds with static data separation, identifying the static data segment using instruction 0xFE, which marks the end of the executable code. The basic blocks not connected to the main graph are removed. Finally, EtherSolve decorates the CFG by identifying and marking the dispatcher (the initial portion of code handling function calls), the entry point of the fallback function, and a unique exit point for terminal basic blocks. These decorations add crucial information for static analysis and vulnerability detection. Once all basic blocks and their connections are identified, the CFG is formed, providing a graphical representation of the code's control flow structure. This CFG construction is represented in Fig. 2 by the second and third blocks.

4.2 Normalization

After creating the CFG, it is essential to extract important information. Only opcode instructions are retained to maintain focus. Line numbers, hexadecimal representations, and all other characters are removed to achieve this. This is crucial because in the next step, the vocabulary present in the graph is analyzed, and it's necessary to avoid any superfluous information that would skew the results. At the end of this phase, a graph with nodes labeled as sequences of opcode instructions is obtained. An example of this node cleaning process is shown in Fig. 3.

4.3 Masked-attention

Once a normalized CFG is created, a masked attention mechanism is applied to obtain more focus on special information.

An attention mechanism is a process designed to identify and prioritize critical aspects to pinpoint potential flaws in the code. However, despite its effectiveness in capturing information, the masked attention mechanism has several limitations. One significant limitation is its tendency to overlook crucial hierarchical relationships within graphs, especially when masks highlight links between nodes (such as during filtering). Moreover, it consistently fails to preserve the traversal order, losing vital structural information. Consequently, despite its proficiency in detecting node similarities, this approach may underestimate critical hierarchical relationships and other essential structures across various contexts.

In contrast, Graph2Vec offers a different approach by generating dense vectors that represent entire graphs. However, this method risks information overload, condensing the whole graph into a single dense vector. This condensation can

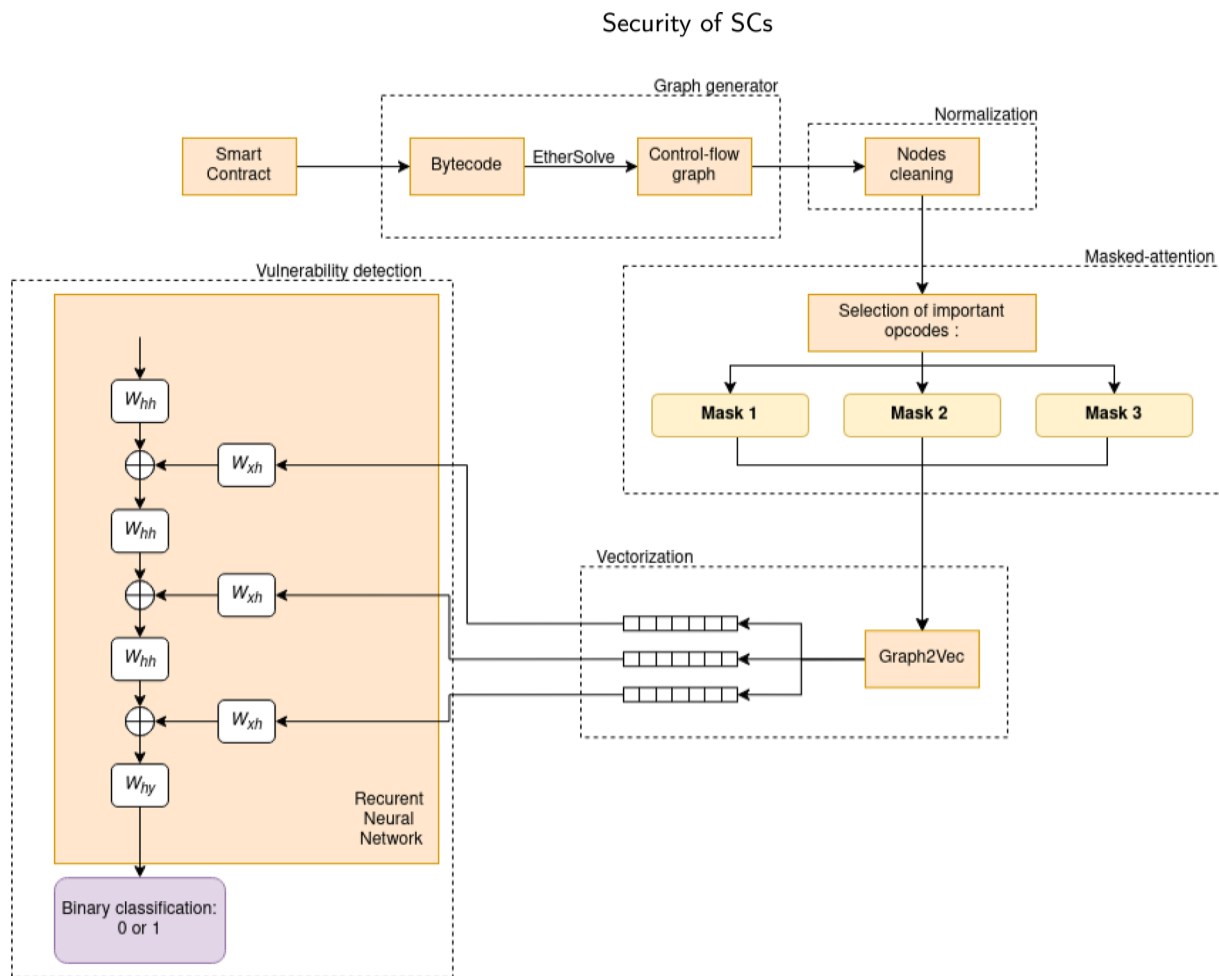


Fig. 2 Proposed architecture

make interpreting the underlying graph structures challenging, as pertinent information may become obscured within a complex array of values.

This research proposes enhancing Graph2Vec by incorporating masks to filter information to address the above limitations. This strategy allows us to segregate and prioritize relevant data, preventing the dilution of critical information among extraneous details. By doing so, we not only improve the efficiency of the graph representation but also enhance the interpretability of the extracted vectors.

Three distinct graphs are constructed by applying masks to the cleaned CFG to focus on the opcodes responsible for vulnerabilities. Each mask selectively retains certain opcodes while excluding others, shown in Table 1 as follows:

- **Mask 1:** Preserves information related to memory management, contract calls, exception handling, and code manipulation. It focuses on detecting vulnerabilities like reentrancy, which is prevalent in SCs.

- **Mask 2:** Retains information on blockchain data retrieval, contract environment access, and time and gas management. It targets vulnerabilities such as timestamp manipulation.
- **Mask 3:** Conserves information regarding arithmetic operations and logic, stack management, and calls. It aims to detect vulnerabilities associated with integer overflow, memory or stack overflow, and incorrect logic.

This approach produces three distinct graphs with identical structures but vastly different contents, allowing us to focus on specific code segments by isolating pertinent information. Each mask strategically isolates specific code characteristics, reducing information overload while preserving critical relationships. Crucially, these masked graphs are processed concurrently - their outputs are dynamically integrated to maintain comprehensive code understanding. This dual mechanism prevents information redundancy by enforcing clear separation of concerns during initial processing and ensures no critical vulnerabilities are overlooked through

Table 1 Details about opcodes in masks

Mask 1	Mask 2	Mask 3
MSTORE	BLOCKHASH	ADD
MLOAD	TIMESTAMP	SUB
SLOAD	NUMBER	MUL
SSTORE	COINBASE	DIV
CALL	GASLIMIT	SDIV
CALLCODE	TIMEDIFF	MOD
DELEGATECALL	GAS	SMOD
STATICCALL	GASPRICE	ADDMOD
CREATE	GASLIMIT	MULMOD
CREATE2	TIMESTAMP	LT
SELFDESTRUCT	BLOCKHASH	GT
CALLER	COINBASE	SLT
CALLVALUE	DIFFICULTY	SGT
CALLDATALOAD		EQ
CALLDATASIZE		ISZERO
CALLDATACOPY		AND
CODESIZE		OR
CODECOPY		XOR
RETURNDATASIZE		NOT
RETURNDATACOPY		BYTE
		SHA3
		PUSH[1-32]
		POP

final cross-mask integration. Figure 4 illustrates the creation of subgraphs based on the masks and the CFG presented in Fig. 3a.

4.4 Vectorization

To prepare data as input for the next step, which is RNN, this research needs to pay attention to the link between nodes and the content of each node. One of the best possibilities is to vectorize each subgraph. A subtle vectorization is a crucial step in preparing graphs for RNN processing. Our selected vectorization approach is Graph2Vec [65]. This algorithm is designed to extract prevalent rooted subgraph patterns from the input graph, thereby capturing key structural motifs essential for analysis. These rooted subgraph patterns are then utilized to train a Machine Learning (ML) algorithm, which learns to predict vector representations. This training process ensures that similar patterns yield comparable representations within the vector space, facilitating more meaningful comparisons and analyses. These learned vector representations aggregate to create a holistic representation of the entire graph, enabling the RNN algorithm to process and analyze the underlying data structure effectively.

4.5 Vulnerability detection

To enhance the efficacy of vulnerability detection methods without using hard parameter sharing or several parallel NNs, we look to leverage the capabilities of RNNs. Unlike traditional models, RNNs can process sequential data while maintaining an internal memory. This innate characteristic empowers them to discern nuanced sequences of relationships embedded within the data sequence under scrutiny. At each sequence step, the RNN assimilates input data alongside its previous state, facilitating a continuous evolution of its internal representation.

Several steps are involved in computing the RNN model's final output. Firstly, during the forward pass, at each step $t \in \{1, 2, 3\}$, the hidden state h_t is computed by applying the Rectified Linear Unit (ReLU) activation function to the weighted sum of the input and the previous hidden state:

$$h_t = \text{ReLU}(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1})$$

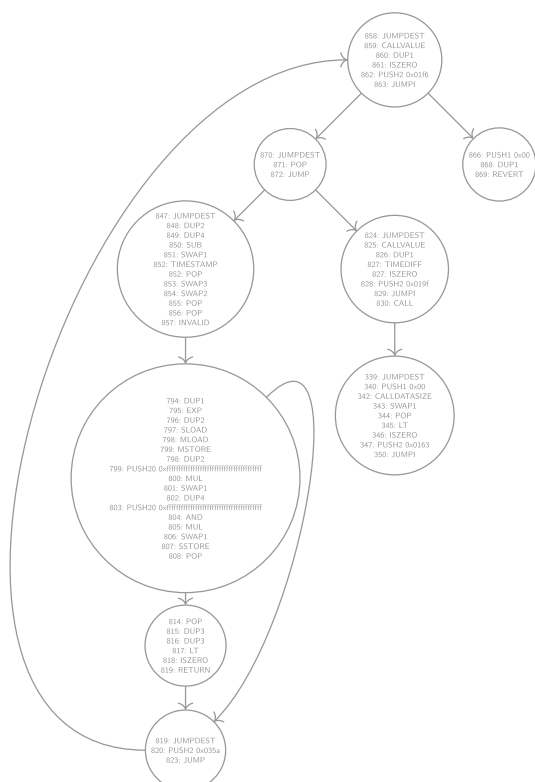
Here, x_t denotes the input vector at step t , W_{xh} represents the weight matrix from the input layer to the hidden layer, and W_{hh} is the weight matrix of the recurrent hidden layer. After processing all time steps, the final output is computed by applying the weight matrix of the hidden layer to the last hidden state, followed by an activation function such as softmax:

$$y = \text{Softmax}(W_{hy} \cdot h_{\text{last}})$$

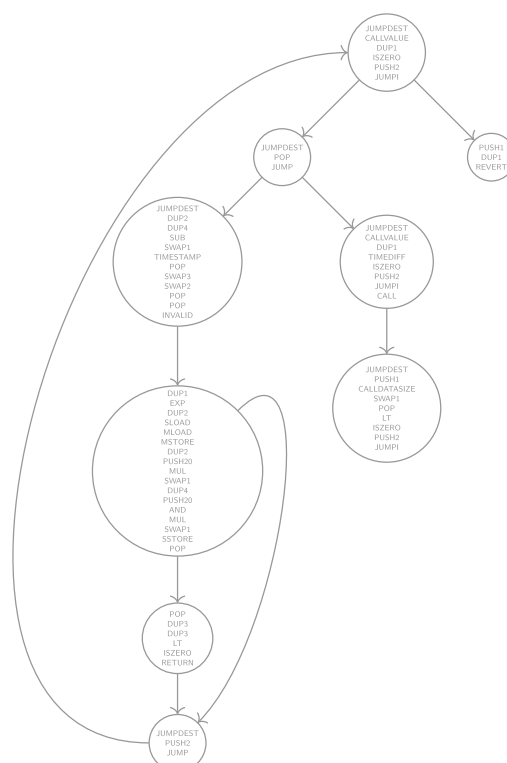
Where h_{last} refers to the hidden state at the previous time step, and W_{hy} represents the weight matrix from the hidden layer to the output layer. In these equations, $\text{ReLU}(\cdot)$ is applied element-wise to a tensor, and $\text{Softmax}(\cdot)$ is used to a vector to obtain a probability distribution over classes. These computations constitute the forward pass of the RNN, culminating in the final output for classification tasks.

This iterative process enables the network to dynamically adapt its understanding as it progresses through the sequence, effectively capturing the intricate nuances of the data. Compared to merely calculating vector similarities, which may overlook subtle contextual cues, this approach allows the model to glean richer information and context from the sequential data. By iteratively refining its representations based on the sequential dependencies, the model can uncover deeper insights and make more informed decisions. The refining leads to enhanced performance and a more comprehensive understanding of the underlying patterns within the data. Additionally, using an RNN allows for the treatment of separate information, which saves resources compared to using different parallel NNs.

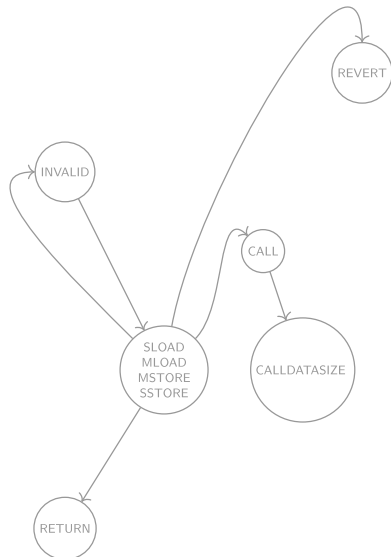
Consequently, the RNN navigates through the complex interconnections within the data, allowing us to uncover vul-



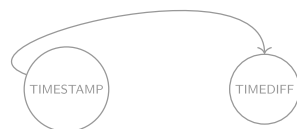
(a) Control flow graph



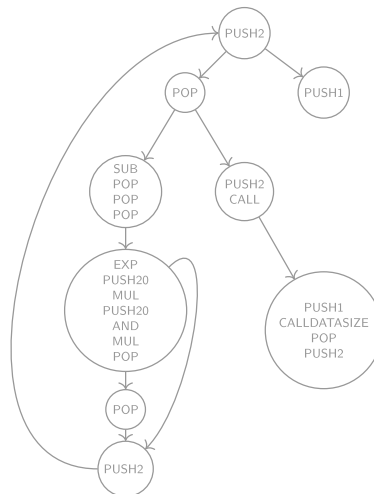
(b) Normalized control flow graph

Fig. 3 Normalization process

(a) Subgraph for mask 1



(b) Subgraph for mask 2



(c) Subgraph for mask 3

Fig. 4 Subgraphs construction

nerabilities concealed within diverse opcode patterns and masks. By seamlessly integrating information across the data graph, the RNN mitigates the risk of overlooking potential vulnerabilities, even amidst unknown threats. With a limited number of sequences to analyze, the RNN proves proficient in preserving crucial information from the initial analysis until the culmination of the detection process. The improvement by a Long Short-Term Memory (LSTM), which could potentially enhance the network's ability to retain long-term dependencies, is therefore unnecessary for our specific task.

5 Experiments and results

The studies referenced in Sect. 3 employ a diverse range of datasets to support their methodologies in detecting vulnerabilities in SCs. Notably, the Ethereum SC Dataset (ESC) and the VNT Chain SC Dataset (VSC) [18, 30] are leveraged to integrate expert-defined security patterns with explainable deep learning models. GasChecker, for example, conducts an empirical analysis of 599,934 contracts deployed on Ethereum to identify patterns of gas inefficiency [16]. Several works utilize Ethereum contracts sourced from platforms such as Etherscan and CVE databases, employing techniques like bytecode normalization and contract-specific slicing for vulnerability detection [11, 12, 16, 17, 20, 30, 32, 40–42, 45, 48, 53, 56, 60]. The SmartBugs Wild dataset is applied with critical data flow graph analysis for vulnerability identification [52, 58]. Additionally, the Majority and Union datasets train deep learning models based on bytecode-level feature vectors [59]. Some datasets include contracts created before September 2018 to support opcode extraction and vulnerability detection through various machine learning techniques [11, 35, 54, 55]. Although these datasets collectively provide a comprehensive foundation for research in SC security, the exact number of contracts per dataset type is not consistently reported.

The dataset used for evaluating our model is the BCCC-VulSCs-2023 dataset, which offers a comprehensive and high-quality benchmark for analyzing vulnerabilities in SCs written in the Solidity programming language [66]. This dataset comprises 34,943 labeled contracts, including 25,903 secure and 9040 vulnerable samples, spanning 12 vulnerability types. BCCC-VulSCs-2023 is carefully curated to ensure the absence of duplicate samples, thereby providing a reliable foundation for robust training and evaluation. Its balanced coverage of common and rare vulnerabilities makes it well-suited for benchmarking detection models. Moreover, since our proposed method leverages CFGs for vulnerability detection, the structural characteristics of the dataset—specifically the diversity in the number of nodes and edges within CFGs—play a critical role in evaluating the scalability and effectiveness of our approach.

The bar chart presented in Fig. 5 provides a visual representation of the distribution of SCs based on the number of nodes and edges. By examining the vertical bars, one can observe how the dataset is distributed across different ranges of graph sizes, ranging from fewer to more nodes or edges. Each bar represents the number of SCs falling into a particular category of graph size, offering a quick insight into the diversity of structures present in the dataset. This granular analysis unveils crucial insights into the complexity and variability of the studied SCs, providing a valuable foundation for understanding dominant structural characteristics and their potential impact on vulnerability detection. Furthermore, the complexity of these structures will be studied in more detail in the subsequent Sect. 6, aiming to elucidate how graph size and topology variations may influence our classification model's performance and robustness.

The repartition reveals several intriguing findings regarding the structure of the examined SCs. Firstly, by observing the ranges of graph sizes, it is noticeable that most SCs exhibit an asymmetric distribution, with a relatively low number of nodes or edges in most cases. The repetition is also really balanced between secure and vulnerable SCs. However, it is also noteworthy to observe the presence of a few SCs with significantly larger graph sizes, indicating significant variation in the complexity and size of the contracts under examination.

To evaluate the model, the dataset is divided into two parts: 80% for training and 20% for testing. This split ensures that the model is trained on a large portion of the data, allowing it to learn and identify patterns effectively. The remaining 20% is used to test the model's performance on unseen data, providing an unbiased assessment of its generalization capabilities. This evaluation method is crucial for understanding how well the model can predict and classify new, unseen instances of secure and vulnerable SCs.

To assess the effectiveness of our vulnerability detection approach, we employ three key metrics: precision, recall, and F1-score. Precision measures the proportion of correctly identified vulnerabilities among all reported cases (true positives/[true positives + false positives]), indicating the tool's answer accuracy. Recall calculates the fraction of actual vulnerabilities successfully detected (true positives/[true positives + false negatives]), reflecting the method's completeness in uncovering vulnerabilities. The F1-score, as the harmonic mean of precision and recall ($2[\text{precision} \times \text{recall}] / [\text{precision} + \text{recall}]$), provides a balanced evaluation that is particularly valuable when dealing with imbalanced datasets common in vulnerability detection. These metrics collectively offer a comprehensive view of detection performance.

Table 2 comprehensively presents the performance of the developed model. Here, we evaluate precision, recall, and F1-score for two distinct SCs classes: Secure and Vulnerable. Additionally, overall performance metrics for the entire

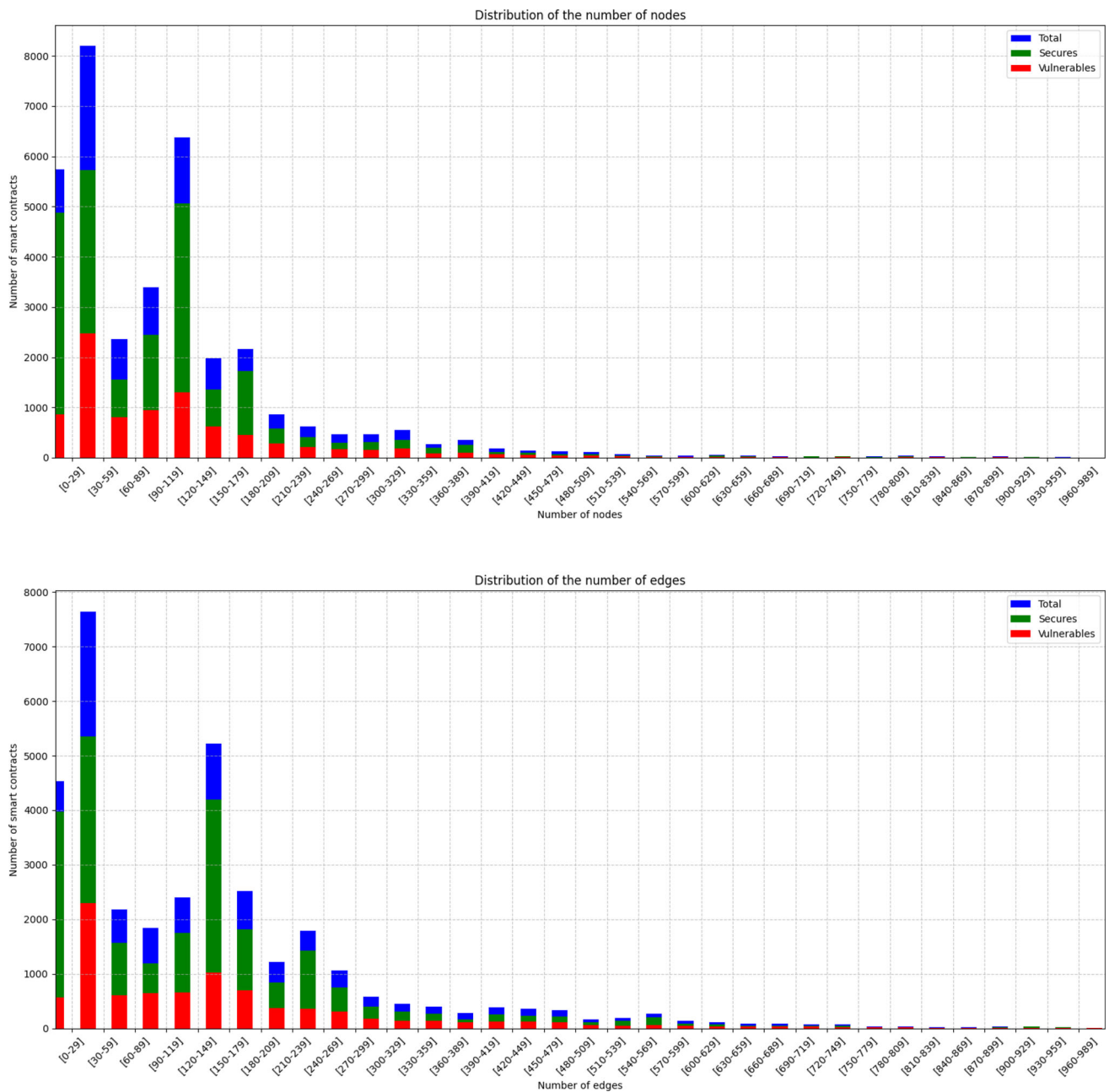


Fig. 5 Size distribution in the used dataset

dataset are provided. These results attest to the effectiveness and reliability of our model for the considered classification task.

In Table 2, zero shows the secure contracts, and one shows the vulnerable ones. As shown, for class 0, which represents secure contracts, the model achieved a precision of 0.92, a recall of 0.98, and an F1-score of 0.95. For class 1, representing vulnerable contracts, the model achieved a precision of 0.98, a recall of 0.94, and an F1-score of 0.96. These high precision and recall values indicate that the model effectively

distinguishes between secure and vulnerable contracts with minimal error.

The model's overall accuracy, which measures the proportion of correct predictions out of all predictions, stands at 0.96. This high accuracy signifies that the model makes accurate predictions for most instances, confirming its robustness.

In addition to accuracy, we calculated the macro average of the performance metrics. The macro average treats all classes equally by averaging their precision, recall, and F1-score without considering the class distribution. The model achieved a macro average precision of 0.95, a recall of 0.96,

Table 2 Classification report

	Precision	Recall	F1-score	Support
0	0.92	0.98	0.95	5000
1	0.98	0.94	0.96	1800
Accuracy			0.96	6800
Macro avg	0.95	0.96	0.96	6800
Weighted avg	0.95	0.96	0.96	6800

and an F1-score of 0.96. These metrics indicate that the model performs uniformly well across both classes without being biased toward the more frequent class.

Furthermore, we computed the weighted average of the performance metrics, which accounts for the number of instances in each class, giving more weight to the larger class. Interestingly, the weighted average precision, recall, and F1-score are identical to the macro averages, all at 0.95, 0.96, and 0.96, respectively. This consistency between macro and weighted averages highlights the model's balanced performance across the different classes of SCs.

In summary, the results presented in Table 2 provide strong evidence of the model's capability to classify secure and vulnerable SCs effectively. The high precision, recall, and F1-score values across both classes, along with the high overall accuracy, macro average, and weighted average metrics, demonstrate the robustness and reliability of our model for this critical classification task. These findings will be further contextualized and compared with existing methodologies in the subsequent Sect. 6, offering more profound insights into the model's performance and implications for SCs security.

6 Analysis and discussion

This section analyzes and discusses the achieved results (Table 2) using three Research Questions (RQs) to thoroughly evaluate the effectiveness of the proposed model in detecting vulnerabilities in SCs. Firstly, RQ1 meticulously analyzes the computational complexity of the model across various stages, providing insights into its efficiency and scalability. The study explains the model's computational requirements by quantifying the complexity involved in CFG construction, vectorization, mask application, and vulnerability detection. Secondly, RQ2 conducts a detailed comparison of the proposed model with existing methods and different NNs architectures. Through rigorous evaluation of precision, recall, and F1-score metrics, the study highlights the competitive edge of the proposed model, demonstrating its superior performance in accurately detecting vulnerabilities. Lastly, RQ3 investigates the impact of preprocessing techniques on the model's performance. The

study underscores the significance of rigorous data preparation in improving vulnerability detection accuracy by examining the efficacy of data preprocessing steps such as node cleaning and masked attention. Through these comprehensive analyses, this study provides valuable insights into the capabilities and potential of the proposed model in enhancing SC vulnerability detection.

(1) RQ1: What is the complexity of the proposed model in terms of time and space ?

To analyze the complexity of the proposed model, the complexity of the CFG construction, the vectorization, the mask application, and the vulnerability detection, the final time complexity is given by Formula (1). In contrast, the space complexity is given by Formula (2). Firstly, the time complexity for constructing a CFG from an SCs bytecode is $O(b_i + n_i + m_i)$. The complexity of the CFG construction for the dataset is

$$O\left(\sum_{i \in [1, k]} n_i + \sum_{i \in [1, k]} m_i + \sum_{i \in [1, k]} b_i\right)$$

where n_i , m_i , and b_i are, respectively, the number of nodes, the number of edges, and the size of the bytecode for the SCs number i .

The next step is the construction of the subgraphs with the application of the mask. The complexity of the step is in $O(n_i + m_i)$ for the SCs number i . This step involves filtering the nodes and removing the edges linked to empty nodes. The global complexity of this step is

$$O\left(\sum_{i \in [1, k]} n_i + \sum_{i \in [1, k]} m_i\right)$$

For the vectorization part, Graph2Vec begins with a feature extraction. This complexity is in $O(n_i + m_i)$ for each graph. Then, in the embedding step, the model creates a vector using the selected features to represent each graph. The total time complexity of the vectorization is dominated by the complexity of feature extraction and model training, and is

$$O\left(\sum_{i \in [1, k]} n_i + \sum_{i \in [1, k]} m_i + W \times \log d_{\max} \sum_{i \in [1, k]} m_i + k \times d \times ep\right)$$

where $d_{\max}$ is the maximum degree of a node in the graphs, k is the number of SCs in the dataset, and d is the dimension of the vector to represent a graph. W is the number of iterations WL, and ep is the number of epochs, which are intrinsic parameters of the model.

The last step of the proposed model is the vulnerability detection with an RNN in $O(l \times d \times k)$, with l the number of parameters in the model. So, the global time complexity

of the model is (Formula 1):

$$O\left(\sum_{i \in [1, k]} n_i + \sum_{i \in [1, k]} m_i + \sum_{i \in [1, k]} b_i + W \times \log d_{\max} \sum_{i \in [1, k]} m_i + k \times d \times ep\right) \quad (1)$$

Secondly, the space complexity for constructing a CFG from an SCs bytecode is $O(n_i + m_i)$. So, the complexity of the CFG construction for all the datasets is

$$O\left(\sum_{i \in [1, k]} n_i + \sum_{i \in [1, k]} m_i\right)$$

The next step is the construction of the subgraphs with the application of the mask. The space complexity of the step is in $O(n_i + m_i)$ for the SCs number i because only nodes and edges of the new subgraphs are conserved. This step involves filtering the nodes and removing the edges linked to empty nodes. The global complexity of this step is

$$O\left(\sum_{i \in [1, k]} n_i + \sum_{i \in [1, k]} m_i\right)$$

For the vectorization part, Graph2Vec begins with a feature selection. This complexity is in $O(d)$ for each graph. Then, in the embedding step, the model creates a vector using the selected features to represent each graph. The total space complexity of the vectorization is dominated by the complexity of feature extraction and model training, and is

$$O\left(W \times \left(\sum_{i \in [1, k]} n_i + \sum_{i \in [1, k]} m_i\right) + k \times d \times ep\right)$$

The last step of the proposed model is the vulnerability detection with an RNN in

$$O(k \times l)$$

So, the global time complexity of the model is (Formula 2):

$$O\left(W \times \left(\sum_{i \in [1, k]} n_i + \sum_{i \in [1, k]} m_i\right) + k \times (d \times ep + l)\right) \quad (2)$$

Based on the presented complexity analysis, the process unfolds through four distinct phases, each contributing uniquely to the computational workload. The polynomial time complexities observed in CFG construction and mask application signify manageable computational demands, which are crucial for establishing the structural representation of SCs and refining subsequent analyses.

However, the vectorization phase introduces significant complexities driven by feature extraction and model training. This phase is particularly resource-intensive due to its

dependence on graph size and model parameters, which are essential for generating representative embeddings of SCs. Efficient optimization of this phase is critical to uphold model performance and manage computational resources effectively.

Additionally, the vulnerability detection phase operates with linear complexity concerning dataset size and model parameters, emphasizing the necessity for robust model architectures and training methodologies. This phase is pivotal for detecting potential security vulnerabilities within SCs, highlighting the importance of accurate detection mechanisms.

Our model maintains strong detection accuracy even when applied to SCs with complex structures and extensive CFGs, demonstrating its scalability and suitability for real-world deployment. It can be seamlessly integrated into blockchain platforms and development environments to perform real-time analysis of SCs, enabling the immediate identification and tagging of vulnerable segments before deployment. This capability is particularly valuable for platforms executing Ethereum-based SCs, where robust security assurance is essential.

Understanding the model's comprehensive complexity profile provides a foundation for optimizing strategies. These strategies are essential for improving computational efficiency and enhancing the model's practical applicability in security contexts, prioritizing scalability and accuracy as critical objectives.

(2) RQ2: How does the proposed model perform compared with existing methods?

Firstly, Table 3 compares our model's effectiveness with the two most updated available solutions. We aim to assess how well our model performs compared to these alternative solutions commonly used in similar tasks. The table comprehensively overviews the key performance metrics such as precision, recall, and F1-score, and it shows that the proposed model achieved the highest scores in the three metrics. This comparison demonstrates the competitive edge and potential advantages of using masks to separate vulnerability information.

A comparison of these three distinct methods for addressing the targeted problem reveals valuable insights. The first two approaches, proposed in [30] and [40], rely on vector representation techniques such as Graph2Vec and Word2Vec, paired with similarity metrics or Convolutional Neural Networks (CNNs). In contrast, the recent methods presented in [67, 68] leverage advanced strategies, including an enhanced genetic algorithm and a heterogeneous graph attention network.

While the first method achieves smaller precision (0.51) compared to the second one (0.61), it has better recall (0.72 versus 0.42), indicating trade-offs between precision and recall in these approaches. The first method's higher recall

Table 3 Comparison with other methods

Reference	Methodology	Precision	Recall	F1-score
[30]	Graph2Vec+Similarity measure	0.51	0.72	0.60
[40]	Word2Vec+CNN	0.61	0.42	0.50
[67]	Enhanced genetic algorithm	0.79	0.59	0.67
[68]	SCVHunter	0.78	0.74	0.76
	Proposed model	0.98	0.94	0.96

suggests it is better at identifying true positives, but its lower precision indicates a higher rate of false positives. Conversely, the second method's higher precision implies better accuracy in its positive predictions, though its lower recall shows it misses many true positives. However, our proposed model stands out distinctly with a precision of 0.98, a recall of 0.94, and an F1 score of 0.96. This indicates that the proposed model minimizes false positives and captures nearly all true positives, achieving an optimal balance between precision and recall. This superior performance underscores the effectiveness of our approach and its potential for practical applications where both high precision and recall are crucial. These results signify the remarkable efficacy of our method in addressing the problem, surpassing the performance of existing approaches.

Secondly, Table 4 presents a comparative analysis of the proposed model tested across different NNs architectures while maintaining consistent preprocessing. By keeping this preprocessing methodology uniform across all experiments, we ensure a fair and direct comparison of the impact of NNs variations on model performance. Each NN architecture introduces unique structural characteristics and complexities, which may influence the model's ability to capture intricate patterns in the data. This systematic evaluation aims to discern the optimal architecture that maximizes performance metrics such as precision and recall.

The comparison among various NNs architectures utilizing the same preprocessing steps sheds light on their respective performances. This study evaluated four architectures: GRU, LSTM, CNN, and RNN. The GRU and LSTM architectures, belonging to the family of RNNs, demonstrate strong precision scores of 0.97 and 0.98, respectively. However, the GRU model exhibits slightly higher recall (0.92) than LSTM (0.85), indicating its ability to capture more relevant instances from the data. Meanwhile, the CNN architecture, known for its effectiveness in capturing spatial dependencies, achieves a precision of 0.61, notably lower than the RNN-based models, while maintaining a respectable recall of 0.80.

Despite these variations, the proposed RNN-based model showcases the performance, boasting a precision of 0.98, a recall of 0.94, and an impressive F1-score of 0.96. These results underscore the superiority of the RNN architecture in

Table 4 Comparison with other NNs

Neural network	Precision	Recall	F1-score
GRU	0.97	0.92	0.94
LSTM	0.98	0.85	0.91
CNN	0.61	0.80	0.69
Proposed model	0.98	0.94	0.96

this particular task, outperforming both the GRU and LSTM models in terms of precision and recall while significantly surpassing the CNN architecture across all metrics. Overall, these findings emphasize the importance of selecting an appropriate NN architecture tailored to the dataset's characteristics and task, with our RNN-based approach as a highly effective solution.

(3) RQ3: How do CFG-based representations, together with advanced preprocessing strategies such as node cleaning and masked attention, affect the accuracy, precision, and recall of semantic graph-based vulnerability detection in SCs?

A foundational component of the proposed model is the extraction of CFGs, which serve as the structural backbone for constructing Semantic Flow Graphs (SFGs). CFGs capture the logical execution paths within an SC, representing control dependencies and branching behavior often linked to critical vulnerabilities such as reentrancy or unchecked calls. Unlike raw source code or token sequences, CFGs offer a structured view of execution semantics, making them particularly suitable for downstream tasks like vulnerability detection. Prior research [46, 63, 64] has shown that CFGs preserve sufficient semantic fidelity to support accurate analysis even when derived from bytecode. By leveraging CFGs as the basis for SFG construction, our model is grounded in a representation that aligns closely with the execution logic of SCs, enabling more effective pattern learning. This theoretical rationale complements the ablation-based evaluation presented in this section, addressing the reviewer's concern by demonstrating the conceptual and empirical value of CFG extraction.

Building upon CFGs, we apply preprocessing strategies to enhance the extracted graphs' semantic quality and improve learning. Data preprocessing is essential for optimizing

model performance, as it involves cleaning and transforming raw inputs to eliminate anomalies, reduce noise, and standardize structural patterns. These steps improve the data's consistency and facilitate more accurate and efficient model training. For instance, preprocessing allows us to handle missing values, encode categorical variables, and normalize feature distributions, which contributes to reducing variance and preventing overfitting in deep learning-based systems.

Within our framework, two key preprocessing components, node cleaning and masked attention, are introduced to refine the graph structure derived from CFGs. Node cleaning eliminates semantically irrelevant or noisy nodes from the graphs, while masked attention mechanisms are applied to emphasize important substructures selectively. These preprocessing steps address specific limitations of deep neural networks, such as sensitivity to irrelevant features and difficulty in learning long-range dependencies. In the following sections, we present an empirical analysis to evaluate the effectiveness of these techniques in improving the model's precision, recall, and overall robustness.

- **Node cleaning:** In our analysis, we compare precision and recall metrics for graphs generated with and without node cleaning to assess its influence on the proposed model's performance (Table 5). This comparison highlights how node cleaning, as a key preprocessing step, affects the quality of graph representations and, by extension, the model's ability to detect vulnerabilities accurately. Notably, the comparison is conducted without applying a masking mechanism in either case, isolating the effect of node cleaning.

The inclusion of node cleaning in the preprocessing pipeline leads to a marked improvement in both recall and precision. Specifically, the model achieves a recall of 0.57 and a precision of 0.77 when node cleaning is applied, demonstrating its effectiveness in filtering out irrelevant or noisy information embedded within the nodes. This refinement allows the model to focus on semantically meaningful content, improving its predictive accuracy. Conversely, omitting node cleaning results in a significant performance drop, with recall falling to just 0.16 and precision declining to 0.65. This stark degradation underscores the adverse impact of noisy or extraneous node content on the model's ability to learn and generalize.

These findings underscore the vital role of preprocessing—particularly node cleaning—in enhancing model robustness and accuracy. By ensuring that only relevant and clean node information is used for graph construction, preprocessing directly contributes to more reliable and effective vulnerability detection outcomes.

Table 5 Utility of node cleaning

	Recall	Precision
With node cleaning	0.57	0.77
Without node cleaning	0.16	0.65

- **Masked-attention:** To propose a high-performance model, we applied different combinations of preprocessing masks. Each combination is tested with the entire graph before the mask is applied. When considering the results presented in Table 6, it becomes evident that the presence of specific masks significantly influences the model's performance.

Firstly, as presented in this table, when the model operates without any masks and only the entire graph, it achieves a moderate level of accuracy (0.67), with corresponding recall and precision values of 0.57 and 0.77, respectively. This serves as a baseline for comparison with other configurations.

The introduction of individual masks or combinations thereof results in varying levels of accuracy, recall, and precision. Notably, certain mask combinations lead to substantial improvements in performance. For instance, the combination of mask 3 and the entire graph greatly increases accuracy to 0.85, with high recall (0.74) and precision (0.95).

Conversely, specific mask configurations exhibit diminished performance, particularly when individual masks are applied in isolation. For instance, using only mask 2, or mask 3 results in lower accuracy and recall values compared to the baseline. However, performance is notably enhanced when these masks are combined with others, such as mask 2 combined with mask 3 or with all masks.

Ultimately, the most effective configuration, marked by the presence of all three masks, leads to the highest accuracy (0.96) and achieves impressive recall (0.94) and precision (0.98). This highlights the synergistic effect of combining multiple masking techniques to optimize model performance.

In summary, the results underscore the importance of carefully selecting and combining masking strategies to enhance the model's robustness and effectiveness, ultimately improving its ability to classify data instances accurately.

Our analysis demonstrates that the proposed model effectively and efficiently detects SCs vulnerabilities. It sets new standards for precision and recall by overcoming complexity challenges and outperforming existing methods. The importance of data preprocessing is also highlighted, showing that

Table 6 Utility of masks

Mask 1	Mask 2	Mask 3	Entire graph	Accuracy	Recall	Precision
			✓	0.67	0.57	0.77
		✓		0.67	0.38	0.91
		✓	✓	0.85	0.74	0.95
	✓			0.57	0.15	0.90
	✓		✓	0.89	0.85	0.94
	✓	✓		0.87	0.78	0.96
	✓	✓	✓	0.92	0.88	0.96
✓				0.79	0.66	0.90
✓			✓	0.88	0.77	0.87
✓		✓		0.89	0.81	0.96
✓		✓	✓	0.84	0.71	0.95
✓	✓			0.93	0.93	0.93
✓	✓		✓	0.92	0.91	0.93
✓	✓	✓	✓	0.93	0.89	0.96
✓	✓	✓		0.96	0.94	0.98

rigorous data preparation significantly enhances the results. These advancements confirm the potential of our approach to strengthen SCs security and encourage further research to optimize these techniques even more.

7 Conclusion and future works

This paper introduces a new method for detecting vulnerabilities in SCs. In contrast to previous approaches, this method uses a vectorization technique to extract information from CFG while utilizing an NNs's processing capacity for graph structure. Simultaneously, a masked-attention mechanism grounded in expert knowledge allocates distinct data into discrete slices. This method investigates the possibility of utilizing graph-based preprocessing with graph vectorization to detect vulnerabilities in SCs. The experimental results on the SC datasets BCCC-VolSCs-2023 showed that the proposed model could effectively improve vulnerable SC detection's accuracy, precision, recall, and F1-score performance of vulnerable SC detection. Specifically, it achieves 0.96 accuracy, 0.98, 0.94, and 0.96 in precision, recall, and F1-score, respectively. Furthermore, the experimentation shows that the pre-processing part has an important role in the model's performance, with an improvement by the masked-attention mechanism of 0.29 for the accuracy, 0.37 for the recall, and 0.11 for the precision. An additional advantage of the proposed method is its full independence from source code. It enables broader applicability in real-world scenarios where only bytecode is accessible, such as in on-chain SC audits.

There are different ways that this work could be developed in the future. First, while this paper mainly focused on separating vulnerable from secure SCs, it would be help-

ful to profile various vulnerabilities. A deeper understanding of different kinds of vulnerabilities could make the profiler more versatile and practical, helping it to address a range of threats better. A deeper understanding of vulnerabilities can allow us to refine the different masks by focusing on precise information. Second, the taxonomy described in this study should be expanded and revised regularly to account for new developments. As technology and security protocols for SCs advance, so should our understanding of them and how to classify them.

Acknowledgements The authors gratefully acknowledge the support of the INTERACTIF SRI program at École Normale Supérieure Paris-Saclay, which provided funding and administrative assistance for the international research internship to Maelle Gautrin, and the grant from Canada Research Chair - Tier II (#CRC-2021-00340) to Arash Habibi Lashkari.

Author Contributions M.G. designed and implemented the model and wrote the main manuscript. A.H.L. supervised the project and reviewed the paper, while S.H. reviewed and revised certain sections.

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no conflict of interest.

References

- Buterin, V., et al.: A next-generation smart contract and decentralized application platform. white paper 3(37), 2–1 (2014)
- Kolvart, M., Poola, M., Rull, A.: Smart contracts. The future of law and etechnologies., 133–147 (2016)

3. Sayeed, S., Marco-Gisbert, H., Caira, T.: Smart contract: attacks and protections. *Ieee Access* **8**, 24416–24427 (2020)
4. Samreen, N.F., Alalfi, M.H.: Reentrancy vulnerability identification in ethereum smart contracts, In: 2020 IEEE International Workshop on Blockchain Oriented Software Engineering (IWBOSE), IEEE, pp. 22–29 (2020)
5. Wang, S., Zhang, C., Su, Z.: Detecting nondeterministic payment bugs in ethereum smart contracts, *Proceedings of the ACM on Programming Languages* 3 (OOPSLA), 1–29 (2019)
6. Luu, L., Chu, D.-H., Olickel, H., Saxena, P., Hobor, A.: Making smart contracts smarter, in: *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, pp. 254–269 (2016)
7. Lee, J.H., et al.: Systematic approach to analyzing security and vulnerabilities of blockchain systems, Ph.D. thesis, Massachusetts Institute of Technology (2019)
8. Lai, E., Luo, W.: Static analysis of integer overflow of smart contracts in ethereum. In: *Proceedings of the 2020 4th International Conference on Cryptography, Security and Privacy*, pp. 110–115 (2020)
9. Khan, Z.A., Namin, A.S.: Ethereum smart contracts: Vulnerabilities and their classifications. In: 2020 IEEE International Conference on big data (big data), IEEE, pp. 1–10 (2020)
10. Zhuang, Y., Liu, Z., Qian, P., Liu, Q., Wang, X., He, Q.: Smart contract vulnerability detection using graph neural networks. In: *Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence*, pp. 3283–3290 (2021)
11. Yu, X., Zhao, H., Hou, B., Ying, Z., Wu, B.: Deesvhunter: A deep learning-based framework for smart contract vulnerability detection. In: 2021 International Joint Conference on Neural Networks (IJCNN), IEEE, pp. 1–8 (2021)
12. Zhang, L., Li, Y., Jin, T., Wang, W., Jin, Z., Zhao, C., Cai, Z., Chen, H.: Spcbig-EC: a robust serial hybrid model for smart contract vulnerability detection. *Sensors* **22**(12), 4621 (2022)
13. Chen, Y., Sun, Z., Gong, Z., Hao, D.: Improving smart contract security with contrastive learning-based vulnerability detection. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pp. 1–11 (2024)
14. Jain, V.K., Tripathi, M.: An integrated deep learning model for Ethereum smart contract vulnerability detection. *Int. J. Inf. Secur.* **23**(1), 557–575 (2024)
15. Liu, Y., Wang, C., Ma, Y.: DI4sc: a novel deep learning-based vulnerability detection framework for smart contracts. *Autom. Softw. Eng.* **31**(1), 24 (2024)
16. Chen, T., Feng, Y., Li, Z., Zhou, H., Luo, X., Li, X., Xiao, X., Chen, J., Zhang, X.: Gaschecker: scalable analysis for discovering gas-inefficient smart contracts. *IEEE Trans. Emerg. Top. Comput.* **9**(3), 1433–1448 (2020)
17. Zhen, Z., Zhao, X., Zhang, J., Wang, Y., Chen, H.: Da-gnn: a smart contract vulnerability detection method based on dual attention graph neural network. *Comput. Netw.* **242**, 110238 (2024)
18. Liu, Z., Qian, P., Wang, X., Zhu, L., He, Q., Ji, S.: Smart contract vulnerability detection: from pure neural network to interpretable graph feature and expert pattern fusion, (2021). [arXiv:2106.09282](https://arxiv.org/abs/2106.09282)
19. Liu, Z., Qian, P., Wang, X., Zhuang, Y., Qiu, L., Wang, X.: Combining graph neural networks with expert knowledge for smart contract vulnerability detection. *IEEE Trans. Knowl. Data Eng.* **35**(2), 1296–1310 (2021)
20. Zhang, L., Wang, J., Wang, W., Jin, Z., Su, Y., Chen, H.: Smart contract vulnerability detection combined with multi-objective detection. *Comput. Netw.* **217**, 109289 (2022)
21. Ghosh, A., Gupta, S., Dua, A., Kumar, N.: Security of cryptocurrencies in blockchain technology: state-of-art, challenges and future prospects. *J. Netw. Comput. Appl.* **163**, 102635 (2020)
22. Khan, S.N., Loukil, F., Ghedira-Guegan, C., Benkhelifa, E., Bani-Hani, A.: Blockchain smart contracts: applications, challenges, and future trends. *Peer-to-peer Netw. Appl.* **14**, 2901–2925 (2021)
23. Alharby, M., Van Moorsel, A.: Blockchain-based smart contracts: A systematic mapping study, (2017). [arXiv:1710.06372](https://arxiv.org/abs/1710.06372)
24. Zhao, Q., Chen, S., Liu, Z., Baker, T., Zhang, Y.: Blockchain-based privacy-preserving remote data integrity checking scheme for IoT information systems. *Inf. Process. Manag.* **57**(6), 102355 (2020)
25. Politou, E., Casino, F., Alepis, E., Patsakis, C.: Blockchain mutability: challenges and proposed solutions. *IEEE Trans. Emerg. Top. Comput.* **9**(4), 1972–1986 (2019)
26. Dutta, P., Choi, T.-M., Somani, S., Butala, R.: Blockchain technology in supply chain operations: applications, challenges and research opportunities. *Transp. Res. Part E Log. Transp. Rev.* **142**, 102067 (2020)
27. Khan, Z.A., Namin, A.S.: A survey on vulnerabilities of ethereum smart contracts, (2020). [arXiv:2012.14481](https://arxiv.org/abs/2012.14481)
28. Krichen, M.: Strengthening the security of smart contracts through the power of artificial intelligence. *Computers* **12**(5), 107 (2023)
29. Perez, D., Livshits, B.: Smart contract vulnerabilities: Does anyone care, 1–15, (2019). [arXiv:1902.06710](https://arxiv.org/abs/1902.06710)
30. Huang, J., Han, S., You, W., Shi, W., Liang, B., Wu, J., Wu, Y.: Hunting vulnerable smart contracts via graph embedding based bytecode matching. *IEEE Trans. Inf. Forensics Secur.* **16**, 2144–2156 (2021)
31. Sun, Y., Wu, D., Xue, Y., Liu, H., Wang, H., Xu, Z., Xie, X., Liu, Y.: Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pp. 1–13, (2024)
32. Guo, J., Lu, L., Li, J.: Smart contract vulnerability detection based on multi-scale encoders. *Electronics* **13**(3), 489 (2024)
33. Yiting, F., Zhaofeng, M., Pengfei, D., Shoushan, L.: Automated vulnerability detection of blockchain smart contracts based on bert artificial intelligent model, *China Communications* (2024)
34. Yang, Z., Zhu, W.: Improvement and optimization of vulnerability detection methods for ethernet smart contracts, *IEEE Access* (2023)
35. Wu, H., Dong, H., He, Y., Duan, Q.: Smart contract vulnerability detection based on hybrid attention mechanism model. *Appl. Sci.* **13**(2), 770 (2023)
36. Zhang, L., Chen, W., Wang, W., Jin, Z., Zhao, C., Cai, Z., Chen, H.: Cbgru: a detection method of smart contract vulnerability based on a hybrid model. *Sensors* **22**(9), 3577 (2022)
37. Torres, C.F., Steichen, M., et al.: The art of the scam: Demystifying honeypots in ethereum smart contracts. In: 28th USENIX Security Symposium (USENIX Security 19), pp. 1591–1607 (2019)
38. Chen, T., Li, X., Luo, X., Zhang, X.: Under-optimized smart contracts devour your money. In: 2017 IEEE 24th international conference on software analysis, evolution and reengineering (SANER), IEEE, pp. 442–446 (2017)
39. Liu, C., Liu, H., Cao, Z., Chen, Z., Chen, B., Roscoe, B.: Reguard: finding reentrancy bugs in smart contracts. In: *Proceedings of the 40th International Conference on Software Engineering: Companion Proceedings*, pp. 65–68 (2018)
40. Sun, Y., Gu, L.: Attention-based machine learning model for smart contract vulnerability detection. In: *Journal of physics: conference series*, Vol. 1820, IOP Publishing, p. 012004 (2021)
41. Sui, J., Chu, L., Bao, H.: An opcode-based vulnerability detection of smart contracts. *Appl. Sci.* **13**(13), 7721 (2023)
42. Huang, J., Zhou, K., Xiong, A., Li, D.: Smart contract vulnerability detection model based on multi-task learning. *Sensors* **22**(5), 1829 (2022)
43. Sender, C., Chen, H., Fereidooni, H., Petzi, L., König, J., Stang, J., Dmitrienko, A., Sadeghi, A.-R., Koushanfar, F.: Smarter contracts:

- Detecting vulnerabilities in smart contracts with deep transfer learning. In: NDSS, p. 2356 (2023)
44. Mezina, A., Ometov, A.: Detecting smart contract vulnerabilities with combined binary and multiclass classification. *Cryptography* **7**(3), 34 (2023)
 45. Huang, J., Zhou, K., Xiong, A., Li, D.: Smart contract vulnerability detection model based on multi-task learning. *Sensors* **22**(5), 1829 (2022)
 46. Contro, F., Crosara, M., Ceccato, M., Dalla Preda, M.: Ethersolve: Computing an accurate control-flow graph from ethereum bytecode. In: 2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC), IEEE, pp. 127–137 (2021)
 47. Chang, J., Gao, B., Xiao, H., Sun, J., Cai, Y., Yang, Z.: scompile: Critical path identification and analysis for smart contracts. In: Formal Methods and Software Engineering: 21st International Conference on Formal Engineering Methods, ICFEM 2019, Shenzhen, China, November 5–9, Proceedings 21, Springer, 2019, pp. 286–304 (2019)
 48. Chen, H., Zhao, X., Wang, Y., Zhen, Z.: Safecheck: Detecting smart contract vulnerabilities based on static program analysis methods. *Security and Privacy* e393 (2024)
 49. Pise, R.G., Patil, S.: Pioneering automated vulnerability detection for smart contracts in blockchain using kevm: Guardian adrgan. *Int. J. Inf. Secur.* 1–15 (2024)
 50. Zeng, Q., He, J., Zhao, G., Li, S., Yang, J., Tang, H., Luo, H.: Ethergis: a vulnerability detection framework for ethereum smart contracts based on graph learning features. In: 2022 IEEE 46th Annual Computers, Software, and Applications Conference (COMPSAC), IEEE, pp. 1742–1749 (2022)
 51. Zhang, L., Wang, J., Wang, W., Jin, Z., Zhao, C., Cai, Z., Chen, H.: A novel smart contract vulnerability detection method based on information graph and ensemble learning. *Sensors* **22**(9), 3581 (2022)
 52. Wu, H., Zhang, Z., Wang, S., Lei, Y., Lin, B., Qin, Y., Zhang, H., Mao, X.: Peculiar: Smart contract vulnerability detection based on crucial data flow graph and pre-training techniques. In: 2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE), IEEE, pp. 378–389 (2021)
 53. Liu, Z., Jiang, M., Zhang, S., Zhang, J., Liu, Y.: A smart contract vulnerability detection mechanism based on deep learning and expert rules. *IEEE Access* **11**, 77990–77999 (2023)
 54. Narayana, K.L., Sathiyamurthy, K.: Automation and smart materials in detecting smart contracts vulnerabilities in blockchain using deep learning. *Mater. Today Proc.* **81**, 653–659 (2023)
 55. Shmatko, O., Kolomiitsev, O., Rekova, N., Kuchuk, N., Matvieiev, O.: Designing and evaluating dl-model for vulnerability detection in smart contracts. *Adv. Inf. Syst.* **7**(4), 41–51 (2023)
 56. Li, P., Li, S., Ding, M., Yu, J., Zhang, H., Zhou, X., Li, J.: A vulnerability detection framework for hyperledger fabric smart contracts based on dynamic and static analysis. In: Proceedings of the 26th International Conference on Evaluation and Assessment in Software Engineering, pp. 366–374 (2022)
 57. Wu, X., Du, X., Yang, Q., Liu, A., Wang, N., Wang, W.: Taintguard: preventing implicit privilege leakage in smart contract based on taint tracking at abstract syntax tree level. *J. Syst. Archit.* **141**, 102925 (2023)
 58. Hwang, S.-J., Choi, S.-H., Shin, J., Choi, Y.-H.: Codenet: code-targeted convolutional neural network architecture for smart contract vulnerability detection. *IEEE Access* **10**, 32595–32607 (2022)
 59. Mi, F., Wang, Z., Zhao, C., Guo, J., Ahmed, F., Khan, L.: Vscl: automating vulnerability detection in smart contracts with deep learning. In: 2021 IEEE International Conference on Blockchain and Cryptocurrency (ICBC), IEEE, 2021, pp. 1–9
 60. Wang, W., Song, J., Xu, G., Li, Y., Wang, H., Su, C.: Contractward: automated vulnerability detection models for Ethereum smart contracts. *IEEE Trans. Netw. Sci. Eng.* **8**(2), 1133–1144 (2020)
 61. Li, P., Wang, G., Xing, X., Li, X., Zhu, J.: Detecting unknown vulnerabilities in smart contracts using opcode sequences. *Connect. Sci.* **36**(1), 2313853 (2024)
 62. Colin, L.S.H., Mohan, P.M., Pan, J., Keong, P.L.K.: An integrated smart contract vulnerability detection tool using multi-layer perceptron on real-time solidity smart contracts. *IEEE Access* **12**, 23549–23567 (2024)
 63. Albert, E., Correias, J., Gordillo, P., Román-Díez, A.H.-C.G., Rubio, A.: Analyzing smart contracts: from evm to a sound control-flow graph, (2020). [arXiv:2004.14437](https://arxiv.org/abs/2004.14437)
 64. Pasqua, M., Benini, A., Contro, F., Crosara, M., Dalla Preda, M., Ceccato, M.: Enhancing Ethereum smart-contracts static analysis by computing a precise control-flow graph of Ethereum bytecode. *J. Syst. Softw.* **200**, 111653 (2023)
 65. Narayanan, A., Chandramohan, M., Venkatesan, R., Chen, L., Liu, Y., Jaiswal, S.: graph2vec: Learning distributed representations of graphs, (2017). [arXiv:1707.05005](https://arxiv.org/abs/1707.05005)
 66. HajiHosseinKhani, S., Lashkari, A.H., Oskui, A.M.: Unveiling vulnerable smart contracts: toward profiling vulnerable smart contracts using genetic algorithm and generating benchmark dataset. *Blockchain Res. Appl.* **5**(1), 100171 (2024)
 67. HajiHosseinKhani, S., Lashkari, A.H., Oskui, A.M.: Unveiling smart contract vulnerabilities: toward profiling smart contract vulnerabilities using enhanced genetic algorithm and generating benchmark dataset. *Blockchain Res. Appl.* **6**(2), 100253 (2025)
 68. Luo, F., Luo, R., Chen, T., Qiao, A., He, Z., Song, S., Jiang, Y., Li, S.: Scvhunter: Smart contract vulnerability detection based on heterogeneous graph attention network. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, pp. 1–13 (2024)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.